

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Schema Analysis and Viva Voce

COURSE NAME: Query Processing for Data
Science

COURSE CODE: DSA0503

COURSE OUTCOME COVERED

CO2: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)

Assessment Tool Weightage: 35%

Dave's Taxonomy: Manipulation (Level 2)
Question Count: 2

Total Marks: 20

Code of Conduct

I Vishwa / 192424180 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Vishwa T

Database Design and Connectivity Rubric

Requirement Analysis (4 Marks)	ER Diagram Design (4 Marks)	Table Design & Normalization (4 Marks)	Database Connectivity using Python (4 Marks)	CRUD Operations (4 Marks)	Total (20 Marks)

Total Marks Awarded:

Question 5: University Course Registration Database

A university maintains the following database schema:

- Student (Student_ID, Student_Name, Department)
- Course (Course_ID, Course_Name, Credits)
- Enrollment (Enrollment_ID, Student_ID, Course_ID, Semester)

Task: Analyze the schema and write a Python program to retrieve the student name, course name, and semester for all students enrolled in courses carrying more than 3 credits.

Aim

To analyze the University Course Registration schema and write a Python program using SQLite to retrieve the student name, course name, and semester for all students enrolled in courses carrying more than 3 credits.

Algorithm

- Step 1: Start.
- Step 2: Import the sqlite3 module and connect to (or create) the database university.db.
- Step 3: Create the Student, Course, and Enrollment tables with appropriate primary and foreign keys as per the schema.
- Step 4: Insert sample records into the Student, Course, and Enrollment tables.
- Step 5: Write a SQL SELECT query that joins Student with Enrollment on Student_ID, and joins the result with Course on Course_ID.
- Step 6: Apply a WHERE clause to filter records where Credits > 3.
- Step 7: Execute the query using cursor.execute() and fetch all rows using fetchall().
- Step 8: Display the Student_Name, Course_Name, and Semester for each fetched row.
- Step 9: Close the database connection.
- Step 10: Stop.

Schema Analysis

The Enrollment table is a bridge (junction) table connecting Student and Course through a many-to-many relationship, since a student can enroll in many courses and a course can have many students. To answer the query, Student is joined with Enrollment on Student_ID, and the result is joined with Course on Course_ID. A WHERE clause then filters courses having Credits > 3.

SQL Query Used

```
SELECT s.Student_Name, c.Course_Name, e.Semester
FROM Student s
JOIN Enrollment e ON s.Student_ID = e.Student_ID
JOIN Course c ON e.Course_ID = c.Course_ID
WHERE c.Credits > 3;
```

Program / Code

```
import sqlite3

# ----- 1. Connect to (or create) the database -----
conn = sqlite3.connect("university.db")
cur = conn.cursor()
```

```

# ----- 2. Create tables as per the schema -----
cur.execute("DROP TABLE IF EXISTS Enrollment")
cur.execute("DROP TABLE IF EXISTS Student")
cur.execute("DROP TABLE IF EXISTS Course")

cur.execute("""
CREATE TABLE Student (
    Student_ID    INTEGER PRIMARY KEY,
    Student_Name  TEXT NOT NULL,
    Department     TEXT
)
""")

cur.execute("""
CREATE TABLE Course (
    Course_ID     INTEGER PRIMARY KEY,
    Course_Name   TEXT NOT NULL,
    Credits       INTEGER NOT NULL
)
""")

cur.execute("""
CREATE TABLE Enrollment (
    Enrollment_ID INTEGER PRIMARY KEY,
    Student_ID    INTEGER,
    Course_ID     INTEGER,
    Semester      TEXT,
    FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
    FOREIGN KEY (Course_ID)  REFERENCES Course(Course_ID)
)
""")

# ----- 3. Insert sample data -----
students = [
    (1, "Aarav Sharma", "CSE"),
    (2, "Diya Patel", "ECE"),
    (3, "Kabir Singh", "CSE"),
    (4, "Meera Nair", "IT"),
]

courses = [
    (101, "Database Management Systems", 4),
    (102, "Operating Systems", 4),
    (103, "Discrete Mathematics", 3),
    (104, "Computer Networks", 3),
    (105, "Machine Learning", 4),
]

enrollments = [
    (1, 1, 101, "Sem 5"),
    (2, 1, 103, "Sem 5"),
    (3, 2, 102, "Sem 5"),
    (4, 2, 105, "Sem 5"),
    (5, 3, 101, "Sem 5"),
    (6, 3, 104, "Sem 5"),
    (7, 4, 105, "Sem 5"),
    (8, 4, 103, "Sem 5"),
]

cur.executemany("INSERT INTO Student VALUES (?, ?, ?)", students)
cur.executemany("INSERT INTO Course VALUES (?, ?, ?)", courses)
cur.executemany("INSERT INTO Enrollment VALUES (?, ?, ?, ?)", enrollments)
conn.commit()

# ----- 4. Query: students enrolled in courses with > 3 credits -----
query = """
SELECT s.Student_Name, c.Course_Name, e.Semester
FROM Student s
JOIN Enrollment e ON s.Student_ID = e.Student_ID
JOIN Course c ON e.Course_ID = c.Course_ID

```

```

WHERE c.Credits > 3;
"""

cur.execute(query)
results = cur.fetchall()

# ----- 5. Display the results -----
print(f"{'Student Name':<18}{'Course Name':<32}{'Semester':<10}")
print("-" * 60)
for student_name, course_name, semester in results:
    print(f"{'student_name':<18}{'course_name':<32}{'semester':<10}")

conn.close()

```

Output

Student Name	Course Name	Semester

Aarav Sharma	Database Management Systems	Sem 5
Diya Patel	Operating Systems	Sem 5
Diya Patel	Machine Learning	Sem 5
Kabir Singh	Database Management Systems	Sem 5
Meera Nair	Machine Learning	Sem 5

Note: Courses with Credits = 3 (Discrete Mathematics, Computer Networks) are correctly excluded since the condition requires Credits strictly greater than 3.

Set B

Question 6: Banking Database

A bank maintains the following database schema:

- Customer (Customer_ID, Customer_Name, City)
- Account (Account_No, Customer_ID, Account_Type, Balance)
- Transaction (Transaction_ID, Account_No, Transaction_Date, Amount)

Task: Analyze the schema and write a Python program to retrieve the customer name, account number, and balance for customers whose account balance exceeds ₹1,00,000.

Aim

To analyze the Banking database schema and write a Python program using SQLite to retrieve the customer name, account number, and balance for customers whose account balance exceeds ₹1,00,000.

Algorithm

- Step 11: Start.
- Step 12: Import the sqlite3 module and connect to (or create) the database bank.db.
- Step 13: Create the Customer, Account, and Transaction tables with appropriate primary and foreign keys as per the schema.
- Step 14: Insert sample records into the Customer, Account, and Transaction tables.
- Step 15: Write a SQL SELECT query that joins Customer with Account on Customer_ID.
- Step 16: Apply a WHERE clause to filter records where Balance > 100000.
- Step 17: Execute the query using cursor.execute() and fetch all rows using fetchall().
- Step 18: Display the Customer_Name, Account_No, and Balance for each fetched row.
- Step 19: Close the database connection.
- Step 20: Stop.

Schema Analysis

Customer has a one-to-many relationship with Account (one customer can hold multiple accounts), and Account has a one-to-many relationship with Transaction. Since the required output only needs customer and balance details, a simple join between Customer and Account on Customer_ID is sufficient; the Transaction table is not required for this particular query. The join result is filtered using WHERE Balance > 100000.

SQL Query Used

```
SELECT c.Customer_Name, a.Account_No, a.Balance
FROM Customer c
JOIN Account a ON c.Customer_ID = a.Customer_ID
WHERE a.Balance > 100000;
```

Program / Code

```
import sqlite3

# ----- 1. Connect to (or create) the database -----
conn = sqlite3.connect("bank.db")
cur = conn.cursor()

# ----- 2. Create tables as per the schema -----
cur.execute("DROP TABLE IF EXISTS Transaction_Tbl")
cur.execute("DROP TABLE IF EXISTS Account")
cur.execute("DROP TABLE IF EXISTS Customer")
```

```

cur.execute("""
CREATE TABLE Customer (
    Customer_ID    INTEGER PRIMARY KEY,
    Customer_Name  TEXT NOT NULL,
    City           TEXT
)
""")

cur.execute("""
CREATE TABLE Account (
    Account_No     INTEGER PRIMARY KEY,
    Customer_ID    INTEGER,
    Account_Type   TEXT,
    Balance        REAL,
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
)
""")

# Note: "Transaction" is a reserved-ish keyword in some contexts,
# so the table is created here as Transaction_Tbl for safety.
cur.execute("""
CREATE TABLE Transaction_Tbl (
    Transaction_ID INTEGER PRIMARY KEY,
    Account_No      INTEGER,
    Transaction_Date TEXT,
    Amount          REAL,
    FOREIGN KEY (Account_No) REFERENCES Account(Account_No)
)
""")

# ----- 3. Insert sample data -----
customers = [
    (1, "Rohan Verma", "Chennai"),
    (2, "Sneha Iyer", "Bengaluru"),
    (3, "Arjun Mehta", "Mumbai"),
    (4, "Priya Reddy", "Hyderabad"),
]

accounts = [
    (5001, 1, "Savings", 150000.00),
    (5002, 1, "Current", 45000.00),
    (5003, 2, "Savings", 85000.00),
    (5004, 3, "Savings", 250000.00),
    (5005, 4, "Current", 120000.00),
]

transactions = [
    (1, 5001, "2026-01-05", 20000.00),
    (2, 5003, "2026-02-10", -5000.00),
    (3, 5004, "2026-03-15", 100000.00),
]

cur.executemany("INSERT INTO Customer VALUES (?, ?, ?)", customers)
cur.executemany("INSERT INTO Account VALUES (?, ?, ?, ?)", accounts)
cur.executemany("INSERT INTO Transaction_Tbl VALUES (?, ?, ?, ?)", transactions)
conn.commit()

# ----- 4. Query: customers with balance > 1,00,000 -----
query = """
SELECT c.Customer_Name, a.Account_No, a.Balance
FROM Customer c
JOIN Account a ON c.Customer_ID = a.Customer_ID
WHERE a.Balance > 100000;
"""

cur.execute(query)
results = cur.fetchall()

# ----- 5. Display the results -----

```

```
print(f"{'Customer Name':<18}{'Account No':<14}{'Balance':<12}")
print("-" * 50)
for customer_name, account_no, balance in results:
    print(f"{'customer_name':<18}{'account_no':<14}{'balance':<12.2f}")

conn.close()
```

Output

Customer Name	Account No	Balance
Rohan Verma	5001	150000.00
Arjun Mehta	5004	250000.00
Priya Reddy	5005	120000.00

Note: Sneha Iyer's account (5003, balance 85000) is excluded as it does not exceed ₹1,00,000. Rohan Verma's current account (5002, balance 45000) is also excluded even though he has another qualifying account, since the filter applies per account, not per customer.